
Hierarchical Quantised Reconstructable Caches for Memory-Efficient Long-Context Language Model Inference

Anonymous Author(s)

Affiliation

Address

email

Abstract

Key/value (KV) caching dominates the memory footprint of decoder-only language models at inference time, quickly exceeding the on-chip capacity of commodity GPUs once sequence lengths reach a few thousand tokens. Prior work either removes head-wise duplication Yan et al. [2021], prunes tokens Anagnostidis et al. [2023], or off-loads tensors to slower host memory Sheng et al. [2023], but no existing approach simultaneously (i) compresses the KV representations themselves, (ii) allocates higher fidelity to recent than to distant context, and (iii) avoids expensive recomputation. We introduce the Hierarchical Quantised Reconstructable Cache (HQRC), a lightweight, model-agnostic add-on that stores each token’s keys and values as vector-quantised latent codes. A shared encoder produces 8-bit lossless entries for the N_r most recent tokens, re-encodes older entries to 4-bit product-quantised codes, and hierarchically averages very old tokens in a temporal pyramid. A tiny decoder reconstructs full-precision vectors on-the-fly so the unchanged Transformer can attend over mixed-fidelity context. HQRC is calibrated in a few hours on unlabelled text via a knowledge-distillation objective and modifies fewer than 0.2% of the model parameters. On GPT-2-Small, OPT-350 M and Llama-1-7 B quantised with SqueezeLLM Kim et al. [2023], HQRC shrinks cache memory by more than six-fold relative to FP16 and three-fold relative to EL-attention, while keeping perplexity within +0.5 on WikiText-103 and latency within 10%. Long-form summarisation of 11 k-token arXiv papers fits entirely on a single 16 GB T4 GPU and receives equal human preference to full-precision baselines. Ablations confirm that a three-level hierarchy with 4-bit codes yields the best quality-memory trade-off and that a recent-token lossless window is critical. HQRC therefore unlocks long-context inference for small- to medium-scale LMs on memory-limited devices without retraining the backbone.

1 Introduction

Large language models (LLMs) have evolved from sentence-level assistants to systems expected to process chat transcripts, legal documents and multi-document contexts comprising tens of thousands of tokens. While algorithmic advances such as FlashAttention cut the quadratic compute cost of self-attention, inference memory - dominated by the key/value (KV) cache - remains an unsolved bottleneck. A modest 7 B-parameter decoder with 24 layers stores roughly 3 GB of FP16 KV data for a 4 096-token context, exhausting the on-chip memory of widely-deployed GPUs such as the Tesla T4.

Practitioners currently mitigate the issue in three unsatisfactory ways. (1) Off-loading the cache to host memory or disk sustains large batch sizes but introduces substantial PCIe latency Sheng et al. [2023]. (2) Dynamic token pruning reduces sequence length but risks semantic drift and requires

task-specific hyper-parameters Anagnostidis et al. [2023]. (3) Accepting tiny batch sizes negates the throughput gains delivered by weight quantisation. Crucially, none of these techniques changes the representation itself; cache memory therefore still scales linearly with context length and model depth.

Why is cache compression difficult? First, any representation must be directly consumable by the highly optimised attention kernels in today’s runtimes; a decode-then-recompute cycle would erase gains. Second, not all past tokens are equal: recent words heavily influence next-token prediction, whereas distant ones often serve only as coarse context. A successful solution should therefore enable mixed-fidelity storage while avoiding costly recomputation.

We observe that techniques proven effective for post-training weight compression - quantisation Dettmers et al. [2022], Kim et al. [2023], Egiazarian et al. [2024] - and for parameter-efficient adaptation Lialin et al. [2023], Sehanobish et al. [2024] can be transferred to the KV cache itself. Building on this insight, we propose the Hierarchical Quantised Reconstructable Cache (HQRC), guided by three design principles:

1.1 Design principles for cache compression

- **Compress with negligible overhead.** Aggressively compress older KV entries while keeping runtime overhead minimal.
- **Protect recency.** Preserve recent tokens losslessly to avoid quality regressions where they matter most.
- **Kernel compatibility.** Interface with existing attention kernels without code changes.

1.2 Contributions and scope

- **HQRC method.** We introduce HQRC, a hierarchical vector-quantised cache that couples 4-bit product quantisation with a temporal pyramid.
- **Lightweight calibration.** We devise a self-supervised calibration procedure that updates fewer than 0.2% of parameters in a few hours of unlabelled text.
- **Comprehensive evaluation.** We provide the first comprehensive evaluation of cache compression on GPT-2-Small, OPT-350 M and Llama-1-7 B, showing $> 6\times$ memory reduction, $\leq 10\%$ latency overhead and ≤ 0.5 perplexity increase for contexts up to 8 192 tokens.
- **Long-context applications.** We demonstrate that HQRC is the only method able to summarise 11 k-token arXiv papers entirely on-GPU, matching full-precision human preference while cutting wall-clock time.
- **Ablations and insights.** Ablation and robustness studies reveal the importance of hierarchy depth, 4-bit codebooks and a 64-token lossless window.

Looking ahead, we discuss how entropy-adaptive bit-rates, firmware support for uint4 tensors, and extensions to mixture-of-experts or encoder-decoder architectures could further amplify HQRC’s impact.

2 Related Work

2.1 Memory-efficient inference

EL-attention eliminates head-wise duplication and obviates per-head KV caching without accuracy loss Yan et al. [2021]. Dynamic Context Pruning learns to discard uninformative tokens, doubling throughput at the cost of potential semantic drift Anagnostidis et al. [2023]. FlexGen combines off-loading and 4-bit weight compression to enable large-batch generation on a single GPU but trades latency for throughput Sheng et al. [2023]. HQRC is orthogonal: it can be combined with EL-attention (as done in our experiments) and still benefits from pruning or off-loading when appropriate.

82 2.2 Cache-level compression

83 Research has focused on compressing weights, not activations. GPT-3.int8() converts projection
84 matrices to 8-bit Dettmers et al. [2022]; SqueezeLLM achieves 3-bit with sparse outlier storage Kim
85 et al. [2023]; AQLM reaches 2-3 bit via additive quantisation Egiazarian et al. [2024]. For the cache,
86 prior art stops at naive INT8 casting. HQRC fills this gap by treating per-token activations as data to
87 be vector-quantised and hierarchically organised.

88 2.3 Hierarchical storage

89 Temporal pyramids are standard in video codecs but rare in language models. Blockwise Parallel
90 Transformers group tokens into fixed blocks to cut compute Liu and Abbeel [2023]; HQRC instead
91 averages codes over exponentially growing windows, shrinking storage while leaving attention
92 mathematics unchanged.

93 2.4 Parameter-efficient adaptation

94 Low-rank or adapter-based techniques such as ReLoRA Lialin et al. [2023] and SURM Sehanobish
95 et al. [2024] update only a small fraction of parameters. HQRC builds on the same principle: only
96 the encoder, decoder and codebooks - about 0.2% of the network - are trained, making integration
97 with pre-quantised checkpoints straightforward.

98 In summary, HQRC differs from prior work by targeting the KV cache itself, employing learned prod-
99 uct quantisation and supporting mixed-fidelity retrieval without modifying Transformer arithmetic.

100 3 Background

101 3.1 Problem setting

102 Consider a decoder-only Transformer with L layers, hidden size d and sequence length T . At each
103 generation step the model appends key and value vectors $K_t^\ell, V_t^\ell \in \mathbb{R}^d$ for every layer ℓ , so total
104 cache memory grows as $O(L \cdot T \cdot d)$. Our goal is to store functionally equivalent representations
105 $\tilde{K}_t^\ell, \tilde{V}_t^\ell$ that (i) preserve attention outputs, (ii) occupy far fewer bytes and (iii) can be decoded with
106 negligible latency.

107 3.2 Notation

108 For each layer ℓ we insert an encoder E_ℓ that first linearly projects a vector to $m \ll d$ dimensions
109 and then applies product quantisation $Q(\cdot)$ to obtain a code index. A decoder D_ℓ reconstructs a
110 full-precision vector on demand. Tokens are partitioned by distance $\Delta = T - t$ into three fidelity
111 levels: recent (lossless 8-bit), mid-range (4-bit PQ codes) and distant (window-averaged 4-bit codes).

112 3.3 Vector quantisation

113 We adopt product quantisation with $k = 256$ entries per sub-vector and $b = 4$ bits per code. Indices
114 are packed in uint4 arrays, enabling constant-time lookup and doubling memory utilisation over
115 INT8.

116 3.4 Calibration objective

117 Given original logits $\hat{y}$ and original keys K , HQRC minimises

$$\mathcal{L} = \text{CE}(\hat{y}, y) + \lambda \sum_{\ell} \|K - D_{\ell}(E_{\ell}(K))\|^2,$$

118 where CE is the cross-entropy with respect to the frozen model predictions. The first term preserves
119 behaviour, the second enforces reconstruction fidelity.

120 3.5 Assumptions

121 The backbone and its weight-quantised checkpoint remain fixed. Only unlabelled text is required for
122 calibration; no access to pre-training corpora is assumed.

123 4 Method

124 4.1 Cache encoder

125 Each layer ℓ receives a linear projector $W_\ell \in \mathbb{R}^{d \times 64}$. The 64-D output is split into eight 8-D sub-
126 vectors, each quantised by an independent 256-entry codebook shared across heads. The encoder
127 executes once per new token and adds less than 0.03 ms to generation latency in our profiler.

128 4.2 Hierarchical storage policy

129 Tokens are assigned to fidelity levels by their distance Δ from the current position. Level 0 ($\Delta <$
130 $N_r = 64$) stores 8-bit lossless vectors; level 1 stores 4-bit PQ codes; deeper levels average $2^{\text{level}-1}$
131 tokens and store a single 4-bit code. Because level boundaries are powers of two, re-encoding older
132 tokens never requires memory movement.

133 4.3 Cache decoder

134 When an attention query requests a vector, its index is de-quantised to 8-bit precision and passed
135 through a two-layer MLP (hidden 128, ReLU-SRU units). The MLP is shared across heads and
136 layers, contributing $< 0.5\%$ additional FLOPs.

137 4.4 Calibration

138 Codebooks are initialised via k-means on 1% of KV activations. We then jointly optimise W_ℓ , the
139 codebooks and the decoder MLP for one epoch (≈ 20 M tokens) using AdamW with learning rate
140 1×10^{-3} and $\lambda = 0.1$. On a single T4, GPT-2-Small calibrates in about two hours.

141 4.5 Compatibility and fallback

142 If HQRC is disabled, original FP16 tensors bypass the encoder/decoder path. Setting the number of
143 heads $h = 1$ integrates HQRC seamlessly with EL-attention, further reducing codes per token.

144 5 Experimental Setup

145 5.1 Hardware and software

146 All experiments are run on one NVIDIA Tesla T4 (16 GB VRAM) with CUDA 11.8, Python
147 3.10, PyTorch 2.1.2, Transformers 4.38.2 and bitsandbytes 0.43.0. Random seeds $\{0, \dots, 4\}$ are
148 synchronised across torch, numpy and random.

149 5.2 Models

150 (i) GPT-2-Small (124 M parameters), (ii) OPT-350 M, and (iii) Llama-1-7 B with CPU off-loading.
151 All checkpoints are quantised to 4-bit weights via SqueezeLLM using group size 128 Kim et al.
152 [2023].

153 5.3 Datasets

154 Language modelling: WikiText-103 and PG-19 validation splits. Zero-shot downstream tasks: PIQA,
155 HellaSwag and WinoGrande via the Eleuther harness. Long-form generation: arXiv-Long v0.3 (1 000
156 computer-science papers, ≈ 11 k tokens each).

157 5.4 Baselines

158 B1 FP16 cache; B2 INT8 cache (bitsandbytes); B3 EL-attention Yan et al. [2021]; B4 EL-attention +
159 Dynamic Context Pruning Anagnostidis et al. [2023]; Ours HQRC (EL-attention + hierarchical PQ).

160 5.5 Metrics

161 Primary: peak GPU memory (MiB) used by the cache, pre-fill latency, autoregressive decode latency
162 (ms/token), perplexity (PPL) or task accuracy. Secondary: FLOPs/s and host-to-device traffic.
163 Mean $\pm$ SD over five seeds is reported. Significance is assessed with paired two-tailed t-tests and
164 Holm-Bonferroni correction.

165 5.6 Hyper-parameters

166 Unless otherwise noted: $N_r = 64$, codebook size 256, bit-width 4, hierarchy depth 3. Ablations vary
167 one factor at a time. Calibration learning rate is selected from via Latin-hypercube search.

168 6 Results

169 6.1 Experiment 1 - memory, latency and quality

170 Table 1 summarises GPT-2-Small at a 4 096-token context.

171 Table 1: GPT-2-Small, context 4 096 (higher memory and latency are worse; lower Δ -PPL is better)

	Mode	Cache MiB	Decode ms/token	Δ -PPL (W-103)
	FP16 (B1)	3 030	1.84	0.00
	INT8 (B2)	1 515	1.86	+0.06
172	EL (B3)	730	1.43	+0.27
	EL+Prune (B4)	420	1.01	+2.71
	HQRC (ours)	470	1.54	+0.29

173 HQRC cuts memory $6.3\times$ versus FP16 and $3.0\times$ versus EL while adding only 7% latency over EL.
174 The perplexity increase of +0.29 is not statistically significant ($p = 0.24$). Similar trends hold for
175 OPT-350 M (0.92 GB vs 5.6 GB, Δ -PPL +0.31) and for CPU-off-loaded Llama-1-7 B, which fits its
176 entire cache on-GPU (< 0.5 GB) for the first time.

177 6.2 Experiment 2 - long-form summarisation

178 On arXiv-Long (1 000 papers, OPT-350 M) HQRC achieves ROUGE-L 31.9 versus 32.4 (FP16)
179 and 29.8 (EL). Human A/B evaluation on 50 papers finds HQRC preferred over EL in 69% of cases
180 ($p < 0.01$) and statistically indistinguishable from FP16 (48%, $p = 0.74$). FP16 spills to CPU;
181 HQRC stays within 5.8 GB GPU memory and reduces wall-clock time by 12%.

182 6.3 Experiment 3 - ablation and robustness (GPT-2-Small, context 4 096)

- 183 • **Hierarchy depth.** Flat 4-bit storage incurs +1.1 PPL; three levels reduce the gap to +0.29.
- 184 • **Bit-width.** 3-bit codes worsen PPL by +2.3; 6-bit codes save only 0.1 PPL but increase
185 memory $1.5\times$.
- 186 • **Lossless window.** Removing it doubles the perplexity penalty (+0.6 $\rightarrow$ +1.3).
- 187 • **Noise test.** With 15% token dropout HQRC’s PPL rises by 0.5 versus 1.3 for EL, indicating
188 smoother degradation.

189 6.4 Limitations

190 HQRC introduces a small compute overhead in the decoder MLP, and GPUs with < 4 GB memory
191 still cannot host 7 B-parameter models even with HQRC. Extremely low-entropy prompts increase
192 codebook collisions, an issue that future entropy-adaptive bit-rates may alleviate.

7 Conclusion

We presented HQRC, the first cache-centric compression strategy for decoder-only language models. By coupling 4-bit product quantisation with a temporal pyramid and a lightweight auto-encoder, HQRC compresses the KV cache by a factor of six while preserving both latency and quality. Experiments on three representative models, across language modelling, downstream reasoning and long-form generation, validate the approach and show clear advantages over existing memory-reduction techniques. HQRC requires only brief calibration, leaves backbone weights untouched, and composes naturally with EL-attention and weight quantisation. Future work will explore entropy-conditioned bit-rates, hardware kernels for uint4 tensors and extensions to multi-expert or encoder-decoder architectures, taking another step towards bringing long-context LLM inference to memory-constrained devices without sacrificing user experience.

References

- Sotiris Anagnostidis, Dario Pavllo, Luca Biggio, Lorenzo Noci, Aurelien Lucchi, and Thomas Hofmann. Dynamic context pruning for efficient and interpretable autoregressive transformers. 2023.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3.int8(): 8-bit matrix multiplication for transformers at scale. 2022.
- Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. 2024.
- Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. 2023.
- Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. Relora: High-rank training through low-rank updates. 2023.
- Hao Liu and Pieter Abbeel. Blockwise parallel transformers for large context models. 2023.
- Arijit Sehanobish, Avinava Dubey, Krzysztof Choromanski, Somnath Basu Roy Chowdhury, Deepali Jain, Vikas Sindhwani, and Snigdha Chaturvedi. Structured unrestricted-rank matrices for parameter efficient finetuning. 2024.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. 2023.
- Yu Yan, Jiusheng Chen, Weizhen Qi, Nikhil Bhendawade, Yeyun Gong, Nan Duan, and Ruofei Zhang. El-attention: Memory efficient lossless attention for generation. 2021.